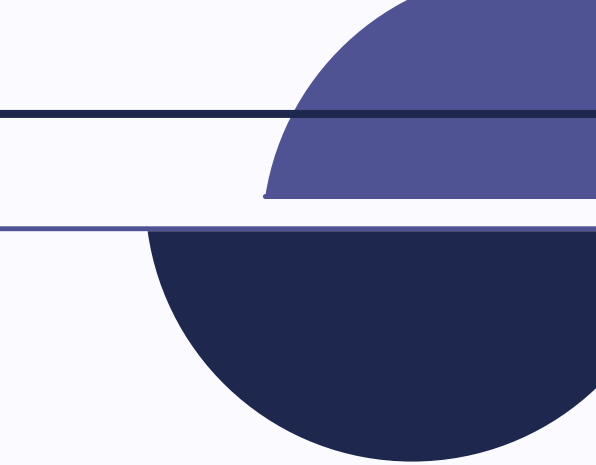




# ANALYTICS BI

*SPRINT 3 PPT*

Group 8



# SPRINT 3 USER STORIES

01

## Chart Visualization

- Creation & Management: Build charts from data
- Rendering Engine: Render interactive visuals
- Customization: Customize chart appearance
- Validation Engine: Validate chart configurations
- Persistence: Save and reuse charts

02

## Dashboard

- Dashboard Gallery
- Interactive Dashboard Editor
- Annotations
- Saving & Persistence
- Metadata editing

03

## Optimization

- Streaming File upload
- Async Processing with Queues
- Error handling & Retrying
- Query Performance
- Bulk Processing

# BACKEND INFRASTRUCTURE & DATA LAYER

- **Async Job System:** Redis + BullMQ queues, retries, dead-letter queue, and worker orchestration.
- **Data Source API:** Dataset schema exposure for chart creation, with fields classified as dimensions and measures.
- **Chart & Dashboard Persistence:** Chart config mapper and annotation persistence.
- **Scoped & Reliable Relationship Linking:** User selects datasets during upload for targeted relationship detection, while manual links are preserved as the source of truth, ensuring efficient and stable data handling.

# VALIDATION, INTEGRITY & PERFORMANCE

- Created chart validation engine enforcing correct data-type usage across all chart types.
- Designed robust data layer: flexible chart schema + schema validation utilities.
- Built reliability systems: idempotent uploads and advanced DLQ with error aggregation & bulk fixes.
- Optimized performance via indexing strategy and centralized index initialization.

# APIS& STREAMING

- Implemented streaming uploads using Busboy enabling large file handling without memory issues.
- Designed API layer: PATCH-based dashboard updates and full chart CRUD with filtering/pagination.
- Integrated real-time system using Socket.io for upload + processing progress tracking.
- Optimized query execution with index-aware MongoDB aggregation pipelines.





---

# QUERY & DATA PIPELINE

- Built dashboard validation and persistent state.
  - Developed frontend-backend integration: UI-to-query conversion system for charts.
  - Refactored ingestion pipeline: async uploads, chunk handling, and job-based processing.
  - Implemented data pipeline: batch inserts, failure handling
- 
-



Thank  
you!